atabase.

3. Initial Cells deployment - Transforming monolithic architecture to Cells architecture

The differences compared to Development Cells are:

- A Cluster-wide Data Provider is introduced by Cells.
- The Cluster-wide Data Provider is deployed with Cell 1 to be able to access cluster-wide data directly.
- The Cluster-wide database is isolated from the main PostgreSQL database.
- A Cluster-wide Data Provider is responsible for storing and sharing user data,
  user sessions (currently stored in Redis sessions cluster), routing information,
  and cluster-wide settings across all Cells.
- Access to the cluster-wide database is done asynchronously:
  - Read access always uses a database replica.
  - A database replica might be deployed with the Cell.
  - Write access uses the dedicated Cluster-wide Data Provider service.
- Additional Cells are deployed, upgraded and maintained via a GitLab Dedicated-like control plane.
- Each Cell aims to run as many services as possible in isolation.
- A Cell can run its own Gitaly cluster, or can use a shared Gitaly cluster, or both.
  Read more in !131657.
- Shared Runners provided by GitLab are expected to be run locally on the Cell.
- Infrastructure components might be shared across the cluster and be used by different Cells.
- It is undefined whether Elasticsearch service would be better run cluster-wide or Cell-local.
- Delay the decision how to scale the GitLab Pages - gitlab.io component.
- Delay the decision how to scale the Registry - registry.gitlab.com component.

4. Hybrid Cells deployment - Initial complete Cells architecture

The differences compared to Initial Cells deployment are:

- Removes coupling of Cell N to Cell 1.
- The Cluster-wide Data Provider is isolated from Cell 1.
- The cluster-wide databases (PostgreSQL, Redis) are moved to be run with the Cluster-wide Data Provider.
- All application data access paths to cluster-wide data use the Cluster-wide Data Provider.
- Some services are shared across Cells.

5. Target Cells - Fully isolated Cells architecture

The differences compared to Hybrid Cells deployment are:

- The Routing Service is expanded to support GitLab Pages and GitLab container registry.

- Each Cell has all services isolated.
- It is allowed that some Cells will follow a hybrid architecture.

Isolation of Services

Each service can be considered individually regarding its requirements, the risks associated with scaling the service, its location (cluster-wide or Cell-local), and impact on our ability to migrate data between Cells.

Cluster-wide services

Service	Type	Uses	Description
Routing Service	GitLab-built	Cluster-wide Data Provider	A general purpose routing service that can redirect requests from all GitLab SaaS domains to the Cell
Cluster-wide Data Provider	GitLab-built	PostgreSQL, Redis, Event Queue?	Provide user profile and routing information to all clustered services

As per the architecture, the above services are required to be run cluster-wide:

- Those are additional services that are introduced by the Cells architecture.

Cell-local services

Service	Type	Uses	Migrate from cluster-wide to Cell	Description
Redis Cluster	Managed service	Disk storage	No problem	Redis is used to hold user sessions, application caches, or Sidekiq queues. Most of that data is only applicable to Cells.
GitLab Runners Manager	Managed service	API, uses Google Cloud VM Instances	No problem	Significant changes required to API and execution of CI jobs

As per the architecture, the above services are required to be run Cell-local:

- The consumer data held by the Cell-local services needs to be migratable to another Cell.
- The compute generated by the service is substantial, and it is strongly desired to reduce impact of single Cell failure.
- It is complex to run the service cluster-wide from the Cells architecture perspective.

Hybrid Services

Service	Type	Uses	Migrate from cluster-wide to Cell	Description
GitLab Pages	GitLab-built	Routing Service, Rails API	No problem	

| Serving CI generated pages under .gitlab.io or custom domains |
GitLab Registry	GitLab-built	Object Storage, PostgreSQL	Non-trivial data migration in case of split	Service to provide GitLab container registry
Gitaly Cluster	GitLab-built	Disk storage, PostgreSQL	No problem: Built-in migration routines to balance Gitaly nodes	Gitaly holds Git repository data. Many Gitaly clusters can be configured in application.
Elasticsearch	Managed service	Many nodes required by sharding	Time-consuming: Rebuild cluster from scratch	Search across all projects
Object Storage	Managed service		Not straightforward: Rather hard to selectively migrate between buckets	Holds all user and CI uploaded files that is served by GitLab

As per the architecture, the above services are allowed to be run either cluster-wide or Cell-local:

- The ability to run hybrid services cluster-wide might reduce the amount of work to migrate data between Cells due to some services being shared.
- The hybrid services that are run cluster-wide might negatively impact Cell availability and resiliency due to increased impact caused by single Cell failure.

Service	Type	Uses	Migrate from cluster-wide to Cell	Description
Elasticsearch	Managed service	Many nodes requires by sharding	Time-consuming: Rebuild cluster from scratch	Search across all projects
Object Storage	Managed service		Not straightforward: Rather hard to selectively migrate between buckets	Holds all user and CI uploaded files that is served by GitLab

As per the architecture, the above services are allowed to be run either cluster-wide or Cell-local:

- The ability to run above services cluster-wide might reduce the amount of work to migrate data between Cells due to some services being shared.
- The hybrid services that are run cluster-wide might negatively impact Cell availability and resiliency due to increased impact caused by single Cell failure.

SECTION: engineering/architecture/design-documents/cells/rejected/proposal-stateless-router-with-buffering-requests.md

{{< engineering/design-document-header >}}

This proposal was superseded by the routing service proposal

{{% alert %}}
This document is a work-in-progress and represents a very early state of the

Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.
{{% /alert %}}

We will decompose gitlabusers, gitlabroutes and gitlabadmin related tables so that they can be shared between all cells and allow any cell to authenticate a user and route requests to the correct cell. Cells may receive requests for the resources they don't own, but they know how to redirect back to the correct cell.

The router is stateless and does not read from the routes database which means that all interactions with the database still happen from the Rails monolith. This architecture also supports regions by allowing for low traffic databases to be replicated across regions.

Users are not directly exposed to the concept of Cells but instead they see different data dependent on their chosen Organization.
Organizations will be a new entity introduced to enforce isolation in the application and allow us to decide which request routes to which Cell, since an Organization can only be on a single Cell.

Differences

The main difference between this proposal and the one with learning routes is that this proposal always sends requests to any of the Cells. If the requests cannot be processed,
the requests will be bounced back with relevant headers. This requires that request to be buffered.
It allows that request decoding can be either via URI or Body of request by Rails.
This means that each request might be sent more than once and be processed more than once as result.

The with learning routes proposal requires that
routable information is always encoded in URI, and the router sends a pre-flight request.

Summary in diagrams

This shows how a user request routes via DNS to the nearest router and the router chooses a cell to send the request to.

```
mermaid
graph TD;
    user((User));
    dns[DNS];
    routerus(Router);
    routereu(Router);
    cellus0{Cell US0};
```

```
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    user-->dns;
    dns-->routerus;
    dns-->routereu;
    subgraph Europe
    routereu-->celleu0;
    routereu-->celleu1;
    end
    subgraph United States
    routerus-->cellus0;
    routerus-->cellus1;
    end
```

<details><summary>More detail</summary>

This shows that the router can actually send requests to any cell. The user will get the closest router to them geographically.

```
mermaid
graph TD;
    user((User));
    dns[DNS];
    routerus(Router);
    routereu(Router);
    cellus0{Cell US0};
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    user-->dns;
    dns-->routerus;
    dns-->routereu;
    subgraph Europe
    routereu-->celleu0;
    routereu-->celleu1;
    end
    subgraph United States
    routerus-->cellus0;
    routerus-->cellus1;
    end
    routereu-.->cellus0;
    routereu-.->cellus1;
    routerus-.->celleu0;
    routerus-.->celleu1;
```

</details>

<details><summary>Even more detail</summary>

This shows the databases. gitlabusers and gitlabroutes exist only in the US region but are replicated to other regions. Replication does not have an arrow because it's too hard to read the diagram.

```
mermaid
graph TD;
    user((User));
    dns[DNS];
    routerus(Router);
    routereu(Router);
    cellus0{Cell US0};
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    dbgitlabusers[(gitlabusers Primary)];
    dbgitlabroutes[(gitlabroutes Primary)];
    dbgitlabusersreplica[(gitlabusers Replica)];
    dbgitlabroutesreplica[(gitlabroutes Replica)];
    dbcellus0[(gitlabmain/gitlabci Cell US0)];
    dbcellus1[(gitlabmain/gitlabci Cell US1)];
    dbcelleu0[(gitlabmain/gitlabci Cell EU0)];
    dbcelleu1[(gitlabmain/gitlabci Cell EU1)];
    user-->dns;
    dns-->routerus;
    dns-->routereu;
    subgraph Europe
    routereu-->celleu0;
    routereu-->celleu1;
    celleu0-->dbcelleu0;
    celleu0-->dbgitlabusersreplica;
    celleu0-->dbgitlabroutesreplica;
    celleu1-->dbgitlabusersreplica;
    celleu1-->dbgitlabroutesreplica;
    celleu1-->dbcelleu1;
    end
    subgraph United States
    routerus-->cellus0;
    routerus-->cellus1;
    cellus0-->dbcellus0;
    cellus0-->dbgitlabusers;
    cellus0-->dbgitlabroutes;
    cellus1-->dbgitlabusers;
    cellus1-->dbgitlabroutes;
    cellus1-->dbcellus1;
    end
    routereu-.->cellus0;
    routereu-.->cellus1;
    routerus-.->celleu0;
```

```
    routerus-.->celleu1;
```

</details>

Summary of changes

1. Tables related to User data (including profile settings, authentication credentials, personal access tokens) are decomposed into a gitlabusers schema
1. The routes table is decomposed into gitlabroutes schema
1. The applicationsettings (and probably a few other instance level tables) are decomposed into gitlabadmin schema
1. A new column routes.cellid is added to routes table
1. A new Router service exists to choose which cell to route a request to.
1. A new concept will be introduced in GitLab called an organization and a user can select a "default organization" and this will be a user level setting. The default organization is used to redirect users away from ambiguous routes like /dashboard to organization scoped routes like /organizations/my-organization/-/dashboard. Legacy users will have a special default organization that allows them to keep using global resources on Cell US0. All existing namespaces will initially move to this public organization.
1. If a cell receives a request for a routes.cellid that it does not own it returns a 302 with X-Gitlab-Cell-Redirect header so that the router can send the request to the correct cell. The correct cell can also set a header X-Gitlab-Cell-Cache which contains information about how this request should be cached to remember the cell. For example if the request was /gitlab-org/gitlab then the header would encode /gitlab-org/ => Cell US0 (for example, any requests starting with /gitlab-org/ can always be routed to Cell US0
1. When the cell does not know (from the cache) which cell to send a request to it just picks a random cell within it's region
1. Writes to gitlabusers and gitlabroutes are sent to a primary PostgreSQL server in our US region but reads can come from replicas in the same region. This will add latency for these writes but we expect they are infrequent relative to the rest of GitLab.

Detailed explanation of default organization in the first iteration

All users will get a new column users.defaultorganization which they can control in user settings. We will introduce a concept of the GitLab.com Public organization. This will be set as the default organization for all existing users. This organization will allow the user to see data from all namespaces in Cell US0 (for example, our original GitLab.com instance). This behavior can be invisible to existing users such that they don't even get told when they are viewing a global page like /dashboard that it's even scoped to an organization.

Any new users with a default organization other than GitLab.com Public will have a distinct user experience and will be fully aware that every page they load is only ever scoped to a single organization. These users can never load any global pages like /dashboard and will end up being redirected to /organizations/<DEFAULTORGANIZATION>/-/dashboard. This may also be the case for legacy APIs and such users may only ever be able to use APIs scoped to a organization.

Detailed explanation of Admin Area settings

We believe that maintaining and synchronizing Admin Area settings will be frustrating and painful so to avoid this we will decompose and share all Admin Area settings in the gitlabadmin schema. This should be safe (similar to other shared schemas) because these receive very little write traffic.

In cases where different cells need different settings (for example, the Elasticsearch URL), we will either decide to use a templated format in the relevant applicationsettings row which allows it to be dynamic per cell. Alternatively if that proves difficult we'll introduce a new table called percellapplicationsettings and this will have 1 row per cell to allow setting different settings per cell. It will still be part of the gitlabadmin schema and shared which will allow us to centrally manage it and simplify keeping settings in sync for all cells.

Pros

1. Router is stateless and can live in many regions. We use Anycast DNS to resolve to nearest region for the user.
1. Cells can receive requests for namespaces in the wrong cell and the user still gets the right response as well as caching at the router that ensures the next request is sent to the correct cell so the next request will go to the correct cell
1. The majority of the code still lives in gitlab rails codebase. The Router doesn't actually need to understand how GitLab URLs are composed.
1. Since the responsibility to read and write gitlabusers, gitlabroutes and gitlabadmin still lives in Rails it means minimal changes will be needed to the Rails application compared to extracting services that need to isolate the domain models and build new interfaces.
1. Compared to a separate routing service this allows the Rails application to encode more complex rules around how to map URLs to the correct cell and may work for some existing API endpoints.
1. All the new infrastructure (just a router) is optional and a single-cell self-managed installation does not even need to run the Router and there are no other new services.

Cons

1. gitlabusers, gitlabroutes and gitlabadmin databases may need to be replicated across regions and writes need to go across regions. We need to do an analysis on write TPS for the relevant tables to determine if this is feasible.
1. Sharing access to the database from many different Cells means that they are all coupled at the Postgres schema level and this means changes to the database schema need to be done carefully in sync with the deployment of all Cells. This limits us to ensure that Cells are kept in closely similar versions compared to an architecture with shared services that have an API we control.
1. Although most data is stored in the right region there can be requests

   proxied from another region which may be an issue for certain types
   of compliance.
1. Data in gitlabusers and gitlabroutes databases must be replicated in
   all regions which may be an issue for certain types of compliance.
1. The router cache may need to be very large if we get a wide variety of URLs
   (for example, long tail). In such a case we may need to implement a 2nd level of
   caching in user cookies so their frequently accessed pages always go to the
   right cell the first time.
1. Having shared database access for gitlabusers and gitlabroutes
   from multiple cells is an unusual architecture decision compared to
   extracting services that are called from multiple cells.
1. It is very likely we won't be able to find cacheable elements of a
   GraphQL URL and often existing GraphQL endpoints are heavily dependent on
   ids that won't be in the routes table so cells won't necessarily know
   what cell has the data. As such we'll probably have to update our GraphQL
   calls to include an organization context in the path like
   /api/organizations/<organization>/graphql.
1. This architecture implies that implemented endpoints can only access data
   that are readily accessible on a given Cell, but are unlikely
   to aggregate information from many Cells.
1. All unknown routes are sent to the latest deployment which we assume to be Cell US0.
   This is required as newly added endpoints will be only decodable by latest cell.
   This Cell could later redirect to correct one that can serve the given request.
   Since request processing might be heavy some Cells might receive significant amount
   of traffic due to that.

 Example database configuration

Handling shared gitlabusers, gitlabroutes and gitlabadmin databases, while having dedicated gitlabmain and gitlabci databases should already be handled by the way we use config/database.yml. We should also, already be able to handle the dedicated EU replicas while having a single US primary for gitlabusers and gitlabroutes. Below is a snippet of part of the database configuration for the Cell architecture described above.

<details><summary>Cell US0</summary>

yaml
 config/database.yml
production:
  main:
    host: postgres-main.cell-us0.primary.consul
    loadbalancing:
      discovery: postgres-main.cell-us0.replicas.consul
  ci:
    host: postgres-ci.cell-us0.primary.consul
    loadbalancing:
      discovery: postgres-ci.cell-us0.replicas.consul
  users:
    host: postgres-users-primary.consul
    loadbalancing:

```
      discovery: postgres-users-replicas.us.consul
  routes:
    host: postgres-routes-primary.consul
    loadbalancing:
      discovery: postgres-routes-replicas.us.consul
  admin:
    host: postgres-admin-primary.consul
    loadbalancing:
      discovery: postgres-admin-replicas.us.consul
```

</details>

<details><summary>Cell EU0</summary>

```
yaml
 config/database.yml
production:
  main:
    host: postgres-main.cell-eu0.primary.consul
    loadbalancing:
      discovery: postgres-main.cell-eu0.replicas.consul
  ci:
    host: postgres-ci.cell-eu0.primary.consul
    loadbalancing:
      discovery: postgres-ci.cell-eu0.replicas.consul
  users:
    host: postgres-users-primary.consul
    loadbalancing:
      discovery: postgres-users-replicas.eu.consul
  routes:
    host: postgres-routes-primary.consul
    loadbalancing:
      discovery: postgres-routes-replicas.eu.consul
  admin:
    host: postgres-admin-primary.consul
    loadbalancing:
      discovery: postgres-admin-replicas.eu.consul
```

</details>

Request flows

1. gitlab-org is a top level namespace and lives in Cell US0 in the GitLab.com Public organization
1. my-company is a top level namespace and lives in Cell EU0 in the my-organization organization

Experience for paying user that is part of my-organization

Such a user will have a default organization set to /my-organization and will be unable to load any global routes outside of this organization. They may load other projects/namespaces but their MR/Todo/Issue counts at the top of the page will not be correctly populated in the first iteration. The user will be aware of this limitation.

Goes to /my-company/my-project while logged in

1. User is in Europe so DNS resolves to the router in Europe
1. They request /my-company/my-project without the router cache, so the router chooses randomly Cell EU1
1. Cell EU1 does not have /my-company, but it knows that it lives in Cell EU0 so it redirects the router to Cell EU0
1. Cell EU0 returns the correct response as well as setting the cache headers for the router /my-company/ => Cell EU0
1. The router now caches and remembers any request paths matching /my-company/ should go to Cell EU0

mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant celleu1 as Cell EU1
    user->>routereu: GET /my-company/my-project
    routereu->>celleu1: GET /my-company/my-project
    celleu1->>routereu: 302 /my-company/my-project X-Gitlab-Cell-Redirect={cell:Cell EU0}
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project... X-Gitlab-Cell-Cache={pathprefix:/my-company/}

Goes to /my-company/my-project while not logged in

1. User is in Europe so DNS resolves to the router in Europe
1. The router does not have /my-company/ cached yet so it chooses randomly Cell EU1
1. Cell EU1 redirects them through a login flow
1. Still they request /my-company/my-project without the router cache, so the router chooses a random cell Cell EU1
1. Cell EU1 does not have /my-company, but it knows that it lives in Cell EU0 so it redirects the router to Cell EU0
1. Cell EU0 returns the correct response as well as setting the cache headers for the router /my-company/ => Cell EU0
1. The router now caches and remembers any request paths matching /my-company/ should go to Cell EU0

mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0

```
    participant celleu1 as Cell EU1
    user->>routereu: GET /my-company/my-project
    routereu->>celleu1: GET /my-company/my-project
    celleu1->>user: 302 /users/signin?redirect=/my-company/my-project
    user->>routereu: GET /users/signin?redirect=/my-company/my-project
    routereu->>celleu1: GET /users/signin?redirect=/my-company/my-project
    celleu1->>user: <h1>Sign in...
    user->>routereu: POST /users/signin?redirect=/my-company/my-project
    routereu->>celleu1: POST /users/signin?redirect=/my-company/my-project
    celleu1->>user: 302 /my-company/my-project
    user->>routereu: GET /my-company/my-project
    routereu->>celleu1: GET /my-company/my-project
    celleu1->>routereu: 302 /my-company/my-project X-Gitlab-Cell-Redirect={cell:Cell EU0}
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project... X-Gitlab-Cell-Cache={pathprefix:/my-company/}
```

Goes to /my-company/my-other-project after last step

1. User is in Europe so DNS resolves to the router in Europe
1. The router cache now has /my-company/ => Cell EU0, so the router chooses Cell EU0
1. Cell EU0 returns the correct response as well as the cache header again

```
mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant celleu1 as Cell EU1
    user->>routereu: GET /my-company/my-project
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project... X-Gitlab-Cell-Cache={pathprefix:/my-company/}
```

Goes to /gitlab-org/gitlab after last step

1. User is in Europe so DNS resolves to the router in Europe
1. The router has no cached value for this URL so randomly chooses Cell EU0
1. Cell EU0 redirects the router to Cell US0
1. Cell US0 returns the correct response as well as the cache header again

```
mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant cellus0 as Cell US0
    user->>routereu: GET /gitlab-org/gitlab
    routereu->>celleu0: GET /gitlab-org/gitlab
    celleu0->>routereu: 302 /gitlab-org/gitlab X-Gitlab-Cell-Redirect={cell:Cell US0}
```

routereu->>cellus0: GET /gitlab-org/gitlab
cellus0->>user: <h1>GitLab.org... X-Gitlab-Cell-Cache={pathprefix:/gitlab-org/}

In this case the user is not on their "default organization" so their TODO counter will not include their typical todos. We may choose to highlight this in the UI somewhere. A future iteration may be able to fetch that for them from their default organization.

Goes to /

1. User is in Europe so DNS resolves to the router in Europe
1. Router does not have a cache for / route (specifically rails never tells it to cache this route)
1. The Router choose Cell EU0 randomly
1. The Rails application knows the users default organization is /my-organization, so it redirects the user to /organizations/my-organization/-/dashboard
1. The Router has a cached value for /organizations/my-organization/ so it then sends the request to POD EU0
1. Cell EU0 serves up a new page /organizations/my-organization/-/dashboard which is the same dashboard view we have today but scoped to an organization clearly in the UI
1. The user is (optionally) presented with a message saying that data on this page is only from their default organization and that they can change their default organization if it's not right.

mermaid
sequenceDiagram
participant user as User
participant routereu as Router EU
participant celleu0 as Cell EU0
user->>routereu: GET /
routereu->>celleu0: GET /
celleu0->>user: 302 /organizations/my-organization/-/dashboard
user->>router: GET /organizations/my-organization/-/dashboard
router->>celleu0: GET /organizations/my-organization/-/dashboard
celleu0->>user: <h1>My Company Dashboard... X-Gitlab-Cell-Cache={pathprefix:/organizations/my-organization/}

Goes to /dashboard

As above, they will end up on /organizations/my-organization/-/dashboard as the rails application will already redirect / to the dashboard page.

Goes to private /not-my-company/not-my-project while logged in

(The user doesn't have access because the project or group is private)

1. User is in Europe so DNS resolves to the router in Europe
1. The router knows that /not-my-company lives in Cell US1 so sends the request to this

1. The user does not have access so Cell US1 returns 404

```
mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant cellus1 as Cell US1
    user->>routereu: GET /not-my-company/not-my-project
    routereu->>cellus1: GET /not-my-company/not-my-project
    cellus1->>user: 404
```

Creates a new top level namespace

The user will be asked which organization they want the namespace to belong to.
If they select my-organization then it will end up on the same cell as all
other namespaces in my-organization. If they select nothing we default to
GitLab.com Public and it is clear to the user that this is isolated from
their existing organization such that they won't be able to see data from both
on a single page.

Experience for GitLab team member that is part of /gitlab-org

Such a user is considered a legacy user and has their default organization set to
GitLab.com Public. This is a "meta" organization that does not really exist but
the Rails application knows to interpret this organization to mean that they are
allowed to use legacy global functionality like /dashboard to see data across
namespaces located on Cell US0. The rails backend also knows that the default cell to render any ambiguous
routes like /dashboard is Cell US0. Lastly the user will be allowed to
go to organizations on another cell like /my-organization but when they do the
user will see a message indicating that some data may be missing (for example, the
MRs/Issues/Todos) counts.

Goes to /gitlab-org/gitlab while not logged in

1. User is in the US so DNS resolves to the US router
1. The router knows that /gitlab-org lives in Cell US0 so sends the request
   to this cell
1. Cell US0 serves up the response

```
mermaid
sequenceDiagram
    participant user as User
    participant routerus as Router US
    participant cellus0 as Cell US0
    user->>routerus: GET /gitlab-org/gitlab
    routerus->>cellus0: GET /gitlab-org/gitlab
    cellus0->>user: <h1>GitLab.org... X-Gitlab-Cell-Cache={pathprefix:/gitlab-org/}
```

Goes to /

1. User is in US so DNS resolves to the router in US
1. Router does not have a cache for / route (specifically rails never tells it to cache this route)
1. The Router chooses Cell US1 randomly
1. The Rails application knows the users default organization is GitLab.com Public, so it redirects the user to /dashboards (only legacy users can see /dashboard global view)
1. Router does not have a cache for /dashboard route (specifically rails never tells it to cache this route)
1. The Router chooses Cell US1 randomly
1. The Rails application knows the users default organization is GitLab.com Public, so it allows the user to load /dashboards (only legacy users can see /dashboard global view) and redirects to router the legacy cell which is Cell US0
1. Cell US0 serves up the global view dashboard page /dashboard which is the same dashboard view we have today

```
mermaid
sequenceDiagram
    participant user as User
    participant routerus as Router US
    participant cellus0 as Cell US0
    participant cellus1 as Cell US1
    user->>routerus: GET /
    routerus->>cellus1: GET /
    cellus1->>user: 302 /dashboard
    user->>routerus: GET /dashboard
    routerus->>cellus1: GET /dashboard
    cellus1->>routerus: 302 /dashboard X-Gitlab-Cell-Redirect={cell:Cell US0}
    routerus->>cellus0: GET /dashboard
    cellus0->>user: <h1>Dashboard...
```

Goes to private project /my-company/my-other-project while logged in

They get a 404.

Experience for non-authenticated users

Flow is similar to authenticated users except global routes like /dashboard will redirect to the login page as there is no default organization to choose from.

A new customers signs up

They will be asked if they are already part of an organization or if they'd like to create one. If they choose neither they end up no the default GitLab.com Public organization.

An organization is moved from 1 cell to another

TODO

GraphQL/API requests which don't include the namespace in the URL

TODO

The autocomplete suggestion functionality in the search bar which remembers recent issues/MRs

TODO

Global search

TODO

Administrator

Loads /admin page

1. Router picks a random cell Cell US0
1. Cell US0 redirects user to /admin/cells/cellus0
1. Cell US0 renders an Admin Area page and also returns a cache header to cache /admin/cellss/cellus0/ => Cell US0. The Admin Area page contains a dropdown list showing other cells they could select and it changes the query parameter.

Admin Area settings in Postgres are all shared across all cells to avoid divergence but we still make it clear in the URL and UI which cell is serving the Admin Area page as there is dynamic data being generated from these pages and the operator may want to view a specific cell.

More Technical Problems To Solve

Replicating User Sessions Between All Cells

Today user sessions live in Redis but each cell will have their own Redis instance. We already use a dedicated Redis instance for sessions so we could consider sharing this with all cells like we do with gitlabusers PostgreSQL database. But an important consideration will be latency as we would still want to mostly fetch sessions from the same region.

An alternative might be that user sessions get moved to a JWT payload that encodes all the session data but this has downsides. For example, it is difficult to expire a user session, when their password changes or for other reasons, if the session lives in a JWT controlled by the user.

How do we migrate between Cells

Migrating data between cells will need to factor all data stores:

1. PostgreSQL

1. Redis Shared State
1. Gitaly
1. Elasticsearch

Is it still possible to leak the existence of private groups via a timing attack?

If you have router in EU, and you know that EU router by default redirects to EU located Cells, you know their latency (lets assume 10 ms). Now, if your request is bounced back and redirected to US which has different latency (lets assume that roundtrip will be around 60 ms) you can deduce that 404 was returned by US Cell and know that your 404 is in fact 403.

We may defer this until we actually implement a cell in a different region. Such timing attacks are already theoretically possible with the way we do permission checks today but the timing difference is probably too small to be able to detect.

One technique to mitigate this risk might be to have the router add a random delay to any request that returns 404 from a cell.

Should runners be shared across all cells?

We have 2 options and we should decide which is easier:

1. Decompose runner registration and queuing tables and share them across all cells. This may have implications for scalability, and we'd need to consider if this would include group/project runners as this may have scalability concerns as these are high traffic tables that would need to be shared.
1. Runners are registered per-cell and, we probably have a separate fleet of runners for every cell or just register the same runners to many cells which may have implications for queueing

How do we guarantee unique ids across all cells for things that cannot conflict?

This project assumes at least namespaces and projects have unique ids across all cells as many requests need to be routed based on their ID. Since those tables are across different databases then guaranteeing a unique ID will require a new solution. There are likely other tables where unique IDs are necessary and depending on how we resolve routing for GraphQL and other APIs and other design goals it may be determined that we want the primary key to be unique for all tables.

SECTION: engineering/architecture/design-documents/cells/rejected/proposal-stateless-router-with-routes-learning.md

{{< engineering/design-document-header >}}

This proposal was superseded by the routing service proposal

{{% alert %}}
This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.
{{% /alert %}}

We will decompose gitlabusers, gitlabroutes and gitlabadmin related tables so that they can be shared between all cells and allow any cell to authenticate a user and route requests to the correct cell. Cells may receive requests for the resources they don't own, but they know how to redirect back to the correct cell.

The router is stateless and does not read from the routes database which means that all interactions with the database still happen from the Rails monolith. This architecture also supports regions by allowing for low traffic databases to be replicated across regions.

Users are not directly exposed to the concept of Cells but instead they see different data dependent on their chosen Organization.
Organizations will be a new entity introduced to enforce isolation in the application and allow us to decide which request routes to which Cell, since an Organization can only be on a single Cell.

Differences

The main difference between this proposal and one with buffering requests is that this proposal uses a pre-flight API request (/api/v4/internal/cells/learn) to redirect the request body to the correct Cell.
This means that each request is sent exactly once to be processed, but the URI is used to decode which Cell it should be directed.

Summary in diagrams

This shows how a user request routes via DNS to the nearest router and the router chooses a cell to send the request to.

```
mermaid
graph TD;
    user((User));
    dns[DNS];
    routerus(Router);
    routereu(Router);
    cellus0{Cell US0};
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    user-->dns;
```

```
    dns-->routerus;
    dns-->routereu;
    subgraph Europe
    routereu-->celleu0;
    routereu-->celleu1;
    end
    subgraph United States
    routerus-->cellus0;
    routerus-->cellus1;
    end
```

More detail

This shows that the router can actually send requests to any cell. The user will get the closest router to them geographically.

```
mermaid
graph TD;
    user((User));
    dns[DNS];
    routerus(Router);
    routereu(Router);
    cellus0{Cell US0};
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    user-->dns;
    dns-->routerus;
    dns-->routereu;
    subgraph Europe
    routereu-->celleu0;
    routereu-->celleu1;
    end
    subgraph United States
    routerus-->cellus0;
    routerus-->cellus1;
    end
    routereu-.->cellus0;
    routereu-.->cellus1;
    routerus-.->celleu0;
    routerus-.->celleu1;
```

Even more detail

This shows the databases. gitlabusers and gitlabroutes exist only in the US region but are replicated to other regions. Replication does not have an arrow because it's too hard to read the diagram.

```
mermaid
graph TD;
    user((User));
    dns[DNS];
    routerus(Router);
    routereu(Router);
    cellus0{Cell US0};
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    dbgitlabusers[(gitlabusers Primary)];
    dbgitlabroutes[(gitlabroutes Primary)];
    dbgitlabusersreplica[(gitlabusers Replica)];
    dbgitlabroutesreplica[(gitlabroutes Replica)];
    dbcellus0[(gitlabmain/gitlabci Cell US0)];
    dbcellus1[(gitlabmain/gitlabci Cell US1)];
    dbcelleu0[(gitlabmain/gitlabci Cell EU0)];
    dbcelleu1[(gitlabmain/gitlabci Cell EU1)];
    user-->dns;
    dns-->routerus;
    dns-->routereu;
    subgraph Europe
    routereu-->celleu0;
    routereu-->celleu1;
    celleu0-->dbcelleu0;
    celleu0-->dbgitlabusersreplica;
    celleu0-->dbgitlabroutesreplica;
    celleu1-->dbgitlabusersreplica;
    celleu1-->dbgitlabroutesreplica;
    celleu1-->dbcelleu1;
    end
    subgraph United States
    routerus-->cellus0;
    routerus-->cellus1;
    cellus0-->dbcellus0;
    cellus0-->dbgitlabusers;
    cellus0-->dbgitlabroutes;
    cellus1-->dbgitlabusers;
    cellus1-->dbgitlabroutes;
    cellus1-->dbcellus1;
    end
    routereu-.->cellus0;
    routereu-.->cellus1;
    routerus-.->celleu0;
    routerus-.->celleu1;
```

Summary of changes

1. Tables related to User data (including profile settings, authentication credentials,

personal access tokens) are decomposed into a gitlabusers schema
1. The routes table is decomposed into gitlabroutes schema
1. The applicationsettings (and probably a few other instance level tables) are decomposed into gitlabadmin schema
1. A new column routes.cellid is added to routes table
1. A new Router service exists to choose which cell to route a request to.
1. If a router receives a new request it will send /api/v4/internal/cells/learn?method=GET&pathinfo=/group-org/project to learn which Cell can process it
1. A new concept will be introduced in GitLab called an organization
1. We require all existing endpoints to be routable by URI, or be fixed to a specific Cell for processing. This requires changing ambiguous endpoints like /dashboard to be scoped like /organizations/my-organization/-/dashboard
1. Endpoints like /admin would be routed always to the specific Cell, like cell0
1. Each Cell can respond to /api/v4/internal/cells/learn and classify each endpoint
1. Writes to gitlabusers and gitlabroutes are sent to a primary PostgreSQL server in our US region but reads can come from replicas in the same region. This will add latency for these writes but we expect they are infrequent relative to the rest of GitLab.

Pre-flight request learning

While processing a request the URI will be decoded and a pre-flight request will be sent for each non-cached endpoint.

When asking for the endpoint GitLab Rails will return information about the routable path. GitLab Rails will decode pathinfo and match it to an existing endpoint and find a routable entity (like project). The router will treat this as short-lived cache information.

1. Prefix match: /api/v4/internal/cells/learn?method=GET&pathinfo=/gitlab-org/gitlab-test/-/issues

```
json
{
  "path": "/gitlab-org/gitlab-test",
  "cell": "cell0",
  "source": "routable"
}
```

1. Some endpoints might require an exact match: /api/v4/internal/cells/learn?method=GET&pathinfo=/-/profile

```
json
{
  "path": "/-/profile",
  "cell": "cell0",
  "source": "fixed",
  "exact": true
}
```

Detailed explanation of default organization in the first iteration

All users will get a new column users.defaultorganization which they can control in user settings. We will introduce a concept of the GitLab.com Public organization. This will be set as the default organization for all existing users. This organization will allow the user to see data from all namespaces in Cell US0 (ie. our original GitLab.com instance). This behavior can be invisible to existing users such that they don't even get told when they are viewing a global page like /dashboard that it's even scoped to an organization.

Any new users with a default organization other than GitLab.com Public will have a distinct user experience and will be fully aware that every page they load is only ever scoped to a single organization. These users can never load any global pages like /dashboard and will end up being redirected to /organizations/<DEFAULTORGANIZATION>/-/dashboard. This may also be the case for legacy APIs and such users may only ever be able to use APIs scoped to a organization.

Detailed explanation of Admin Area settings

We believe that maintaining and synchronizing Admin Area settings will be frustrating and painful so to avoid this we will decompose and share all Admin Area settings in the gitlabadmin schema. This should be safe (similar to other shared schemas) because these receive very little write traffic.

In cases where different cells need different settings (eg. the Elasticsearch URL), we will either decide to use a templated format in the relevant applicationsettings row which allows it to be dynamic per cell. Alternatively if that proves difficult we'll introduce a new table called percellapplicationsettings and this will have 1 row per cell to allow setting different settings per cell. It will still be part of the gitlabadmin schema and shared which will allow us to centrally manage it and simplify keeping settings in sync for all cells.

Pros

1. Router is stateless and can live in many regions. We use Anycast DNS to resolve to nearest region for the user.
1. Cells can receive requests for namespaces in the wrong cell and the user still gets the right response as well as caching at the router that ensures the next request is sent to the correct cell so the next request will go to the correct cell
1. The majority of the code still lives in gitlab rails codebase. The Router doesn't actually need to understand how GitLab URLs are composed.
1. Since the responsibility to read and write gitlabusers, gitlabroutes and gitlabadmin still lives in Rails it means minimal changes will be needed to the Rails application compared to extracting services that need to isolate the domain models and build new interfaces.
1. Compared to a separate routing service this allows the Rails application

to encode more complex rules around how to map URLs to the correct cell and may work for some existing API endpoints.
1. All the new infrastructure (just a router) is optional and a single-cell self-managed installation does not even need to run the Router and there are no other new services.

Cons

1. gitlabusers, gitlabroutes and gitlabadmin databases may need to be replicated across regions and writes need to go across regions. We need to do an analysis on write TPS for the relevant tables to determine if this is feasible.
1. Sharing access to the database from many different Cells means that they are all coupled at the Postgres schema level and this means changes to the database schema need to be done carefully in sync with the deployment of all Cells. This limits us to ensure that Cells are kept in closely similar versions compared to an architecture with shared services that have an API we control.
1. Although most data is stored in the right region there can be requests proxied from another region which may be an issue for certain types of compliance.
1. Data in gitlabusers and gitlabroutes databases must be replicated in all regions which may be an issue for certain types of compliance.
1. The router cache may need to be very large if we get a wide variety of URLs (ie. long tail). In such a case we may need to implement a 2nd level of caching in user cookies so their frequently accessed pages always go to the right cell the first time.
1. Having shared database access for gitlabusers and gitlabroutes from multiple cells is an unusual architecture decision compared to extracting services that are called from multiple cells.
1. It is very likely we won't be able to find cacheable elements of a GraphQL URL and often existing GraphQL endpoints are heavily dependent on ids that won't be in the routes table so cells won't necessarily know what cell has the data. As such we'll probably have to update our GraphQL calls to include an organization context in the path like /api/organizations/<organization>/graphql.
1. This architecture implies that implemented endpoints can only access data that are readily accessible on a given Cell, but are unlikely to aggregate information from many Cells.
1. All unknown routes are sent to the latest deployment which we assume to be Cell US0. This is required as newly added endpoints will be only decodable by latest cell. Likely this is not a problem for the /internal/cells/learn is it is lightweight to process and this should not cause a performance impact.

Example database configuration

Handling shared gitlabusers, gitlabroutes and gitlabadmin databases, while having dedicated gitlabmain and gitlabci databases should already be handled by the way we use config/database.yml. We should also, already be able to handle the dedicated EU replicas while having a single US primary for gitlabusers and gitlabroutes. Below is a snippet of part of the

database configuration for the Cell architecture described above.

Cell US0:

yaml
 config/database.yml
```
production:
  main:
    host: postgres-main.cell-us0.primary.consul
    loadbalancing:
      discovery: postgres-main.cell-us0.replicas.consul
  ci:
    host: postgres-ci.cell-us0.primary.consul
    loadbalancing:
      discovery: postgres-ci.cell-us0.replicas.consul
  users:
    host: postgres-users-primary.consul
    loadbalancing:
      discovery: postgres-users-replicas.us.consul
  routes:
    host: postgres-routes-primary.consul
    loadbalancing:
      discovery: postgres-routes-replicas.us.consul
  admin:
    host: postgres-admin-primary.consul
    loadbalancing:
      discovery: postgres-admin-replicas.us.consul
```

Cell EU0:

yaml
 config/database.yml
```
production:
  main:
    host: postgres-main.cell-eu0.primary.consul
    loadbalancing:
      discovery: postgres-main.cell-eu0.replicas.consul
  ci:
    host: postgres-ci.cell-eu0.primary.consul
    loadbalancing:
      discovery: postgres-ci.cell-eu0.replicas.consul
  users:
    host: postgres-users-primary.consul
    loadbalancing:
      discovery: postgres-users-replicas.eu.consul
  routes:
    host: postgres-routes-primary.consul
    loadbalancing:
      discovery: postgres-routes-replicas.eu.consul
```

```
  admin:
    host: postgres-admin-primary.consul
    loadbalancing:
      discovery: postgres-admin-replicas.eu.consul
```

Request flows

1. gitlab-org is a top level namespace and lives in Cell US0 in the GitLab.com Public organization
1. my-company is a top level namespace and lives in Cell EU0 in the my-organization organization

Experience for paying user that is part of my-organization

Such a user will have a default organization set to /my-organization and will be unable to load any global routes outside of this organization. They may load other projects/namespaces but their MR/Todo/Issue counts at the top of the page will not be correctly populated in the first iteration. The user will be aware of this limitation.

Goes to /my-company/my-project while logged in

1. User is in Europe so DNS resolves to the router in Europe
1. They request /my-company/my-project without the router cache, so the router chooses randomly Cell EU1
1. The /internal/cells/learn is sent to Cell EU1, which responds that resource lives on Cell EU0
1. Cell EU0 returns the correct response
1. The router now caches and remembers any request paths matching /my-company/ should go to Cell EU0

```
mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant celleu1 as Cell EU1
    user->>routereu: GET /my-company/my-project
    routereu->>celleu1: /api/v4/internal/cells/learn?method=GET&pathinfo=/my-company/my-project
    celleu1->>routereu: {path: "/my-company", cell: "celleu0", source: "routable"}
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project...
```

Goes to /my-company/my-project while not logged in

1. User is in Europe so DNS resolves to the router in Europe
1. The router does not have /my-company/ cached yet so it chooses randomly Cell EU1
1. The /internal/cells/learn is sent to Cell EU1, which responds that resource lives on Cell

EU0
1. Cell EU0 redirects them through a login flow
1. User requests /users/signin, uses random Cell to run /internal/cells/learn
1. The Cell EU1 responds with cell0 as a fixed route
1. User after login requests /my-company/my-project which is cached and stored in Cell EU0
1. Cell EU0 returns the correct response

mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant celleu1 as Cell EU1
    user->>routereu: GET /my-company/my-project
    routereu->>celleu1: /api/v4/internal/cells/learn?method=GET&pathinfo=/my-company/my-project
    celleu1->>routereu: {path: "/my-company", cell: "celleu0", source: "routable"}
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: 302 /users/signin?redirect=/my-company/my-project
    user->>routereu: GET /users/signin?redirect=/my-company/my-project
    routereu->>celleu1: /api/v4/internal/cells/learn?method=GET&pathinfo=/users/signin
    celleu1->>routereu: {path: "/users", cell: "celleu0", source: "fixed"}
    routereu->>celleu0: GET /users/signin?redirect=/my-company/my-project
    celleu0-->>user: <h1>Sign in...
    user->>routereu: POST /users/signin?redirect=/my-company/my-project
    routereu->>celleu0: POST /users/signin?redirect=/my-company/my-project
    celleu0->>user: 302 /my-company/my-project
    user->>routereu: GET /my-company/my-project
    routereu->>celleu0: GET /my-company/my-project
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project...

Goes to /my-company/my-other-project after last step

1. User is in Europe so DNS resolves to the router in Europe
1. The router cache now has /my-company/ => Cell EU0, so the router chooses Cell EU0
1. Cell EU0 returns the correct response as well as the cache header again

mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant celleu1 as Cell EU1
    user->>routereu: GET /my-company/my-project
    routereu->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project...

Goes to /gitlab-org/gitlab after last step

1. User is in Europe so DNS resolves to the router in Europe
1. The router has no cached value for this URL so randomly chooses Cell EU0
1. Cell EU0 redirects the router to Cell US0
1. Cell US0 returns the correct response as well as the cache header again

mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    participant cellus0 as Cell US0
    user->>routereu: GET /gitlab-org/gitlab
    routereu->>celleu0: /api/v4/internal/cells/learn?method=GET&pathinfo=/gitlab-org/gitlab
    celleu0->>routereu: {path: "/gitlab-org", cell: "cellus0", source: "routable"}
    routereu->>cellus0: GET /gitlab-org/gitlab
    cellus0->>user: <h1>GitLab.org...

In this case the user is not on their "default organization" so their TODO
counter will not include their typical todos. We may choose to highlight this in
the UI somewhere. A future iteration may be able to fetch that for them from
their default organization.

Goes to /

1. User is in Europe so DNS resolves to the router in Europe
1. Router does not have a cache for / route (specifically rails never tells it to cache this
route)
1. The Router choose Cell EU0 randomly
1. The Rails application knows the users default organization is /my-organization, so
   it redirects the user to /organizations/my-organization/-/dashboard
1. The Router has a cached value for /organizations/my-organization/ so it then sends the
   request to POD EU0
1. Cell EU0 serves up a new page /organizations/my-organization/-/dashboard which is the same
   dashboard view we have today but scoped to an organization clearly in the UI
1. The user is (optionally) presented with a message saying that data on this page is only
   from their default organization and that they can change their default
   organization if it's not right.

mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant celleu0 as Cell EU0
    user->>routereu: GET /
    routereu->>celleu0: GET /
    celleu0->>user: 302 /organizations/my-organization/-/dashboard
    user->>router: GET /organizations/my-organization/-/dashboard
    router->>celleu0: GET /organizations/my-organization/-/dashboard
    celleu0->>user: <h1>My Company Dashboard... X-Gitlab-Cell-

Cache={pathprefix:/organizations/my-organization/}

Goes to /dashboard

As above, they will end up on /organizations/my-organization/-/dashboard as
the rails application will already redirect / to the dashboard page.

Goes to private project or group /not-my-company/not-my-project while logged in

1. User is in Europe so DNS resolves to the router in Europe
1. The router knows that /not-my-company lives in Cell US1 so sends the request to this
1. The user does not have access so Cell US1 returns 404

```
mermaid
sequenceDiagram
    participant user as User
    participant routereu as Router EU
    participant cellus1 as Cell US1
    user->>routereu: GET /not-my-company/not-my-project
    routereu->>cellus1: GET /not-my-company/not-my-project
    cellus1->>user: 404
```

Creates a new top level namespace

The user will be asked which organization they want the namespace to belong to.
If they select my-organization then it will end up on the same cell as all
other namespaces in my-organization. If they select nothing we default to
GitLab.com Public and it is clear to the user that this is isolated from
their existing organization such that they won't be able to see data from both
on a single page.

Experience for GitLab team member that is part of /gitlab-org

Such a user is considered a legacy user and has their default organization set to
GitLab.com Public. This is a "meta" organization that does not really exist but
the Rails application knows to interpret this organization to mean that they are
allowed to use legacy global functionality like /dashboard to see data across
namespaces located on Cell US0. The rails backend also knows that the default cell to render any ambiguous
routes like /dashboard is Cell US0. Lastly the user will be allowed to
go to organizations on another cell like /my-organization but when they do the
user will see a message indicating that some data may be missing (eg. the
MRs/Issues/Todos) counts.

Goes to /gitlab-org/gitlab while not logged in

1. User is in the US so DNS resolves to the US router
1. The router knows that /gitlab-org lives in Cell US0 so sends the request

   to this cell
1. Cell US0 serves up the response

mermaid
sequenceDiagram
    participant user as User
    participant routerus as Router US
    participant cellus0 as Cell US0
    user->>routerus: GET /gitlab-org/gitlab
    routerus->>cellus0: GET /gitlab-org/gitlab
    cellus0->>user: <h1>GitLab.org...

 Goes to /

1. User is in US so DNS resolves to the router in US
1. Router does not have a cache for / route (specifically rails never tells it to cache this route)
1. The Router chooses Cell US1 randomly
1. The Rails application knows the users default organization is GitLab.com Public, so it redirects the user to /dashboards (only legacy users can see /dashboard global view)
1. Router does not have a cache for /dashboard route (specifically rails never tells it to cache this route)
1. The Router chooses Cell US1 randomly
1. The Rails application knows the users default organization is GitLab.com Public, so it allows the user to load /dashboards (only legacy users can see /dashboard global view) and redirects to router the legacy cell which is Cell US0
1. Cell US0 serves up the global view dashboard page /dashboard which is the same dashboard view we have today

mermaid
sequenceDiagram
    participant user as User
    participant routerus as Router US
    participant cellus0 as Cell US0
    participant cellus1 as Cell US1
    user->>routerus: GET /
    routerus->>cellus1: GET /
    cellus1->>user: 302 /dashboard
    user->>routerus: GET /dashboard
    routerus->>cellus1: /api/v4/internal/cells/learn?method=GET&pathinfo=/dashboard
    cellus1->>routerus: {path: "/dashboard", cell: "cellus0", source: "routable"}
    routerus->>cellus0: GET /dashboard
    cellus0->>user: <h1>Dashboard...

 Goes to private project /my-company/my-other-project while logged in

They get a 404.

Experience for non-authenticated users

Flow is similar to logged in users except global routes like /dashboard will redirect to the login page as there is no default organization to choose from.

A new customers signs up

They will be asked if they are already part of an organization or if they'd like to create one. If they choose neither they end up no the default GitLab.com Public organization.

An organization is moved from 1 cell to another

TODO

GraphQL/API requests which don't include the namespace in the URL

TODO

The autocomplete suggestion functionality in the search bar which remembers recent issues/MRs

TODO

Global search

TODO

Administrator

Loads /admin page

1. The /admin is locked to Cell US0
1. Some endpoints of /admin, like Projects in Admin are scoped to a Cell and users needs to choose the correct one in a dropdown, which results in endpoint like /admin/cells/cell0/projects.

Admin Area settings in Postgres are all shared across all cells to avoid divergence but we still make it clear in the URL and UI which cell is serving the Admin Area page as there is dynamic data being generated from these pages and the operator may want to view a specific cell.

More Technical Problems To Solve

Replicating User Sessions Between All Cells

Today user sessions live in Redis but each cell will have their own Redis instance. We already use a dedicated Redis instance for sessions so we could consider sharing this with all cells like we do with gitlabusers PostgreSQL database. But an important consideration will be latency as we would still want to mostly fetch sessions from the same region.

An alternative might be that user sessions get moved to a JWT payload that encodes all the session data but this has downsides. For example, it is difficult to expire a user session, when their password changes or for other reasons, if the session lives in a JWT controlled by the user.

How do we migrate between Cells

Migrating data between cells will need to factor all data stores:

1. PostgreSQL
1. Redis Shared State
1. Gitaly
1. Elasticsearch

Is it still possible to leak the existence of private groups via a timing attack?

If you have router in EU, and you know that EU router by default redirects to EU located Cells, you know their latency (lets assume 10 ms). Now, if your request is bounced back and redirected to US which has different latency (lets assume that roundtrip will be around 60 ms) you can deduce that 404 was returned by US Cell and know that your 404 is in fact 403.

We may defer this until we actually implement a cell in a different region. Such timing attacks are already theoretically possible with the way we do permission checks today but the timing difference is probably too small to be able to detect.

One technique to mitigate this risk might be to have the router add a random delay to any request that returns 404 from a cell.

Should runners be shared across all cells?

We have 2 options and we should decide which is easier:

1. Decompose runner registration and queuing tables and share them across all cells. This may have implications for scalability, and we'd need to consider if this would include group/project runners as this may have scalability concerns as these are high traffic tables that would need to be shared.
1. Runners are registered per-cell and, we probably have a separate fleet of runners for every cell or just register the same runners to many cells which may have implications for queueing

How do we guarantee unique ids across all cells for things that cannot conflict?

This project assumes at least namespaces and projects have unique ids across all cells as many requests need to be routed based on their ID. Since those tables are across different databases then guaranteeing a unique ID will require a new solution. There are likely other tables where unique IDs are necessary and depending on how we resolve routing for GraphQL and other APIs and other design goals it may be determined that we want the primary key to be unique for all tables.

SECTION: engineering/architecture/design-documents/ci_builds_runner_fleet_metrics/_index.md

{{< engineering/design-document-header >}}

The CI section envisions new value-added features in GitLab for CI Builds and Runner Fleet focused on observability and automation. However, implementing these features and delivering on the product vision of observability, automation, and AI optimization using the current database architecture in PostgreSQL is very hard because:

- CI-related transactional tables are huge, so any modification to them can increase the load on the database and subsequently cause incidents.
- PostgreSQL is not optimized for running aggregation queries.
- We also want to add more information from the build environment, making CI tables even larger.
- We also need a data model to aggregate data sets for the GitLab CI efficiency machine learning models - the basis of the Runner Fleet AI solution

We want to create a new flexible database architecture which:

- will support known reporting requirement